

Maelstrom 分布式系统一致性与容错测试床速查海报

M1: Clojure 运行层与内核拓扑

Card 1: Clojure JVM 运行时

Maelstrom 物理内核基于 Clojure 编写，运行在 JVM 之上。它仿真分布式测试拓扑，通过创建子进程拉起待测节点二进制。

内存调优：‘-Xmx4g’ 强制限制 JVM 最大堆内存，防止路径分析引起内存爆炸崩溃。

Card 2: STDIN/STDOUT 文本协议

测试主进程与节点子进程间全部依赖标准输入输出交换 ‘\n’ 分隔的 JSON 报文。

重要约束：待测节点绝不能将未经格式化的调试输出写入 stdout。任何 ‘println’ 都会破坏 JSON 流，导致 Maelstrom 死锁，调试日志必须强定向输出到 stderr。

Card 3: 节点间消息通信

节点以为自身在进行 Socket 网络调用，其实是由 Maelstrom 在内核内存中截获并重定向转发包。

消息格式：包裹 ‘src’、‘dest’ 及 ‘body’。消息可靠性由 Maelstrom 内部路由策略提供单机仿真支持。

Card 4: 状态循环与并发

内部依赖 Clojure Agent/Ref/Atom 无锁同步机制。

并发限界：单机并发处理吞吐受限，主线程并发度与 CPU 核心数绑定，测试建议节点数目控制在 20 以下。

Card 5: JSON Schema RPC

定义 RPC 请求的 ‘msg_id’ 以及响应返回匹配的 ‘in_reply_to’，并在失败时返回对应 ‘error_code’。

协议代价：编解码带来一定的 CPU 消耗，但在系统级跨语言测试中拥有极高的兼容适配性。

M2: Jepsen 核心引擎与测试框架

Card 6: Knossos 线性化校验

基于 WGL (Wing & Gong) 算法验证单 key 执行历史路径，构建全局合法执行排序全序流。

限制调优：并发写冲突发生时路径爆炸，必须使用 ‘-time-limit’ 强行限制搜索最大深度防止 OOM。

Card 7: Elle 事务一致性分析

通过收集多 key 事务生成的 WW、WR、RW 物理依赖图谱，并在图谱中检测是否存在异常环路。

检测边界：完美识别 G1a (脏写)、G-single 等复杂事务一致性违背。

Card 8: Generator 事件生成器

通过 Clojure 事件队列定制压测客户端的行为，可选择高争抢 key 分布、高吞吐混合读写负载等模式。

Card 9: Checker 归纳引擎

测试执行完毕后，分析 ‘history.txt’，计算运行统计指标并产生 ‘results.edn’ 结论文档。

Card 10: 客户端代理物理并发

每一个并发客户端代理在宿主机上对应一个 OS 物理线程，在网络延迟注入下测试系统并发收敛表现。

M3: 一致性模型与定理约束

Card 11: 因果一致性验证

要求有因果先后关联的写操作必须全局一致。

实现机制：在报文首部引入向量时钟 (Vector Clock) 并在客户端持久化水印。

Card 12: 读未提交与已提交

读未提交允许事务读取未提交的临时中间状态值（脏读）；读已提交则通过 MVCC 活动事务列表隔离未提交版本。

Card 13: 可重复读

保证在同一个事务范围的多次读取必然返回一致的值。在物理层通常需要加上 Read-Shared 锁保护直到事务提交。

Card 14: 串行化一致性

要求所有多副本并发事务的执行结果与某种单线程串行执行历史表现等价。Elle 环路判定上完全禁止环路。

Card 15: 单调读写约束

单调读保证同一个会话读取的版本单调不减；单调写强制写操作在主副本串行排队物理生效。

M4: 分布式共享存储与服务

Card 16: Lin-KV 强一致存储

Maelstrom 提供内置的线性一致性 KV 存储接口 ‘lin-kv’。测试节点通过 HTTP 远程存取状态，保证最高一致性。

Card 17: Seq-KV 顺序一致存储

内置的 ‘seq-kv’ 保证全局操作相同的某种偏序，不提供实时一致性约束。其优势在于吞吐性能显著高于 ‘lin-kv’。

Card 18: Lww-Register 冲突合并

基于 Last-Write-Wins 原理收敛值冲突。

致命缺陷：完全依赖节点的物理时钟，在存在时钟偏移时会导致最新的修改被陈旧写入覆盖。

Card 19: PN-Counter 状态合并

支持并发递增递减的 CRDT 计数器，节点各自累加 P 向量和 N 向量，合并时取各个索引的全局最大值。

Card 20: 副本日志同步

分布式多副本在一致性测试中需要自建一致性写入，写入日志在获得 Quorum (‘N/2+1’) 确认后方能正式提交。

M5: 网络混沌注入与状态转换

Card 21: 网络分区混沌注入

Nemesis 将测试网络集群划分为物理隔绝的若干分区，迫使系统进入脑裂和少数派分区拒绝响应状态。

Card 22: 丢包率动态模拟

通过 ‘-drop-rate’ 参数强行控制路由转发失败概率。验证当网络连接线断开时，系统重试和确认重传机制。

Card 23: 时延膨胀与乱序

‘-latency’ 基准延迟及 ‘-delta’ 时延抖动模拟。导致数据包顺序在物理网络管道传输中发生强乱序。

Card 24: Nemesis 故障引入器

Clojure 封装的核心故障控制器，按照预定状态机周期性地执行网络隔离与断网故障注入。

Card 25: 恢复重连与路由恢复

Nemesis 解除故障状态后重建路由，考察待测节点是否能够重连、选举对齐 term 并自动清洗死锁及脏数据。

M6: 诊断调试与测试调优

Card 26: Gnuplot 吞吐时延图

测试完成后，系统利用 Gnuplot 引擎生成直观的 ‘latency-raw.png’ 与 ‘throughput.png’ 图表文件。

Card 27: History 历史日志追踪

检查 ‘store/latest/history.txt’ 每一行包含的操作 ID、读写类型和数值，直接定位违背强一致性的异常事务。

Card 28: 命令行参数调优字典

典型运行命令示范：

```
‘./maelstrom  test  -w  txn-list  -bin
./my_node  -node-count  5  -time-limit
30 -rate 100 -nemesis partition’
```